

Lab Assignment 5
Object-Oriented Programming

Branch
B.E., 2nd Year – ENC



Submitted By:
Kaushtubh Kumar
Roll No. 1024150191
Batch: 2023

Question 1

Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

Code

```
#include <iostream>
using namespace std;

class Base {
public:
    void displayBase() {
        cout << "Base class method" << endl;
    }
};

class Derived : public Base {
public:
    void displayDerived() {
        cout << "Derived class method" << endl;
    }
};

int main() {
    Base b;
    b.displayBase();

    Derived d;
    d.displayBase();    // Inherited from Base
    d.displayDerived(); // Own method

    return 0;
}
```

OUTPUT

```
Base class method
Base class method
Derived class method

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 2

Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

Code

```
#include <iostream>
using namespace std;

class Base {
protected:
    int protectedValue;
public:
    Base() : protectedValue(100) {}
    void setProtectedValue(int val) {
        protectedValue = val;
    }
};

class Derived : public Base {
public:
    void displayProtected() {
        cout << "Protected value: " << protectedValue << endl;
    }
    void modifyProtected(int val) {
        protectedValue = val;
    }
};

int main() {
    Derived d;
    d.displayProtected();
    d.modifyProtected(200);
    d.displayProtected();
    return 0;
}
```

OUTPUT

```
Protected value: 100
Protected value: 200

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 3

Try protected and private access modifiers to understand the difference of various modes of inheritance.

Code

```
#include <iostream>
using namespace std;

class Base {
public:
    int publicVar;
protected:
    int protectedVar;
private:
    int privateVar;
public:
    Base() : publicVar(1), protectedVar(2), privateVar(3) {}
};

class DerivedPublic : public Base {
    // public members stay public
    // protected members stay protected
    // private members not accessible
};

class DerivedProtected : protected Base {
    // public members become protected
    // protected members stay protected
};

class DerivedPrivate : private Base {
    // public members become private
    // protected members become private
};

int main() {
    DerivedPublic dp;
    dp.publicVar = 10; // Accessible
```

```

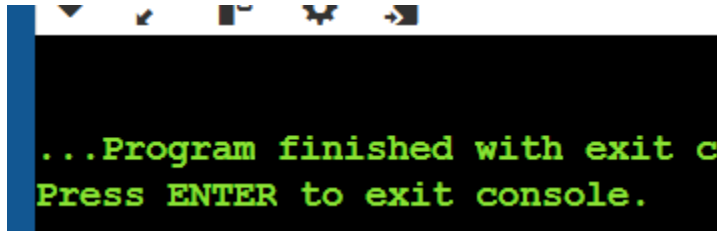
    DerivedProtected dpr;
    // dpr.publicVar = 10; // Error: now protected

    DerivedPrivate dpv;
    // dpv.publicVar = 10; // Error: now private

    return 0;
}

```

OUTPUT



Question 4

Write small C++ codes for each inheritance type (Single, Multiple, Hierarchical, Multilevel, Hybrid).

Code

```

#include <iostream>
using namespace std;

// Single Inheritance
class Animal {
public:
    void eat() { cout << "Eating..." << endl; }
};
class Dog : public Animal {
public:
    void bark() { cout << "Barking..." << endl; }
};

// Multiple Inheritance
class Walk {
public:
    void walk() { cout << "Walking..." << endl; }
};
class Swim {
public:
    void swim() { cout << "Swimming..." << endl; }
};
class Frog : public Walk, public Swim {};

// Hierarchical Inheritance

```

```

class Bird : public Animal {
public:
    void fly() { cout << "Flying..." << endl; }
};
class Fish : public Animal {
public:
    void swim() { cout << "Swimming..." << endl; }
};

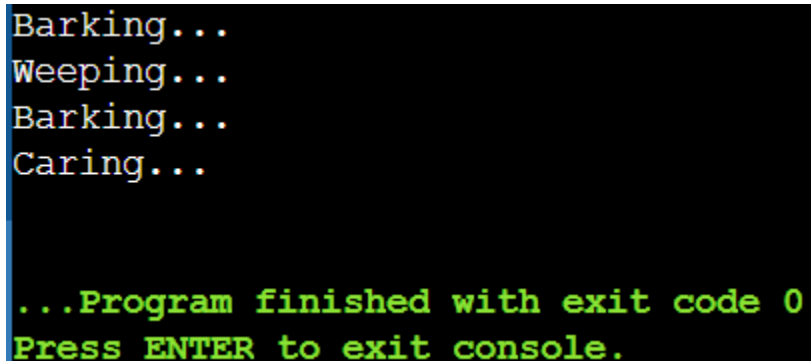
// Multilevel Inheritance
class Puppy : public Dog {
public:
    void weep() { cout << "Weeping..." << endl; }
};

// Hybrid Inheritance
class Pet {
public:
    void care() { cout << "Caring..." << endl; }
};
class PetDog : public Dog, public Pet {};

int main() {
    Dog d; d.eat(); d.bark();
    Frog f; f.walk(); f.swim();
    Bird b; b.eat(); b.fly();
    Puppy p; p.eat(); p.bark(); p.weep();
    PetDog pd; pd.bark(); pd.care();
    return 0;
}

```

OUTPUT



```

Barking...
Weeping...
Barking...
Caring...

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 5

How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

Code

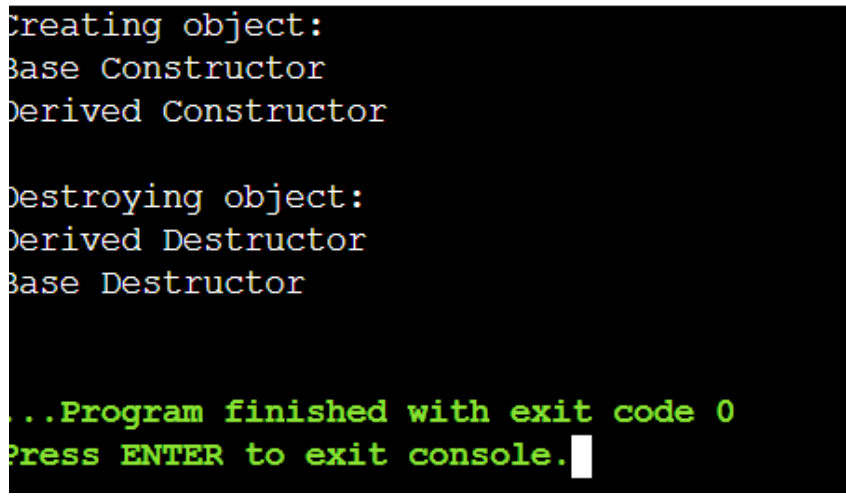
```
#include <iostream>
using namespace std;

class Base {
public:
    Base() { cout << "Base Constructor" << endl; }
    ~Base() { cout << "Base Destructor" << endl; }
};

class Derived : public Base {
public:
    Derived() { cout << "Derived Constructor" << endl; }
    ~Derived() { cout << "Derived Destructor" << endl; }
};

int main() {
    cout << "Creating object:" << endl;
    Derived d;
    cout << "\nDestroying object:" << endl;
    return 0;
}
```

OUTPUT



```
Creating object:
Base Constructor
Derived Constructor

Destroying object:
Derived Destructor
Base Destructor

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 6

Implement a base class Book with attributes title, author, and price. Then, create a derived class Textbook that inherits from Book and adds a new attribute subject.

Code

```
#include <iostream>
#include <string>
using namespace std;

class Book {
protected:
    string title;
    string author;
    double price;
public:
    Book(string t, string a, double p)
        : title(t), author(a), price(p) {}
    void display() {
        cout << "Title: " << title
              << ", Author: " << author
              << ", Price: $" << price << endl;
    }
};

class Textbook : public Book {
private:
    string subject;
public:
    Textbook(string t, string a, double p, string s)
        : Book(t, a, p), subject(s) {}
    void displayTextbook() {
        display();
        cout << "Subject: " << subject << endl;
    }
};

int main() {
    Book b("1984", "George Orwell", 15.99);
    b.display();

    Textbook tb("Physics Vol 1", "H.C. Verma", 29.99, "Physics");
    tb.displayTextbook();

    return 0;
}
```

OUTPUT


```
Title: 1984, Author: George Orwell, Price: $15.99
Title: Physics Vol 1, Author: H.C. Verma, Price: $29.99
Subject: Physics
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Question 7

Implement CarDashboard class that inherits from Speedometer, FuelGauge, and Thermometer (Multiple Inheritance).

Code

```
#include <iostream>
#include <string>
using namespace std;

class Speedometer {
protected:
    int speed;
public:
    Speedometer(int s = 0) : speed(s) {}
    void setSpeed(int s) { speed = s; }
    int getSpeed() { return speed; }
    void displaySpeed() {
        cout << "Speed: " << speed << " km/h" << endl;
    }
};

class FuelGauge {
protected:
    int fuelLevel;
public:
    FuelGauge(int f = 100) : fuelLevel(f) {}
    void setFuelLevel(int f) { fuelLevel = f; }
    int getFuelLevel() { return fuelLevel; }
    void displayFuel() {
        cout << "Fuel Level: " << fuelLevel << "%" << endl;
    }
};

class Thermometer {
protected:
    double temperature;
public:
    Thermometer(double t = 25.0) : temperature(t) {}
    void setTemperature(double t) { temperature = t; }
    double getTemperature() { return temperature; }
    void displayTemp() {
```

```

        cout << "Temperature: " << temperature << " C" << endl;
    }
};

class CarDashboard : public Speedometer, public FuelGauge, public Thermometer {
public:
    CarDashboard(int s, int f, double t)
        : Speedometer(s), FuelGauge(f), Thermometer(t) {}
    void displayAll() {
        cout << "=== Dashboard ===" << endl;
        displaySpeed();
        displayFuel();
        displayTemp();
    }
};

int main() {
    CarDashboard cd(80, 75, 22.5);
    cd.displayAll();
    return 0;
}

```

OUTPUT

```

=== Dashboard ===
Speed: 80 km/h
Fuel Level: 75%
Temperature: 22.5 C

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 8

Implement hierarchical inheritance with LibraryUser base class and Student, Teacher derived classes.

Code

```
#include <iostream>
#include <string>
using namespace std;

class LibraryUser {
protected:
    string name;
    int id;
    string contact;
public:
    LibraryUser(string n, int i, string c)
        : name(n), id(i), contact(c) {}
    void displayUser() {
        cout << "Name: " << name
              << ", ID: " << id
              << ", Contact: " << contact << endl;
    }
};

class Student : public LibraryUser {
private:
    int gradeLevel;
public:
    Student(string n, int i, string c, int g)
        : LibraryUser(n, i, c), gradeLevel(g) {}
    void displayStudent() {
        displayUser();
        cout << "Grade Level: " << gradeLevel << endl;
    }
};

class Teacher : public LibraryUser {
private:
    string department;
public:
    Teacher(string n, int i, string c, string d)
        : LibraryUser(n, i, c), department(d) {}
    void displayTeacher() {
        displayUser();
        cout << "Department: " << department << endl;
    }
};

int main() {
    Student s("John", 1001, "john@email.com", 10);
    s.displayStudent();
}
```

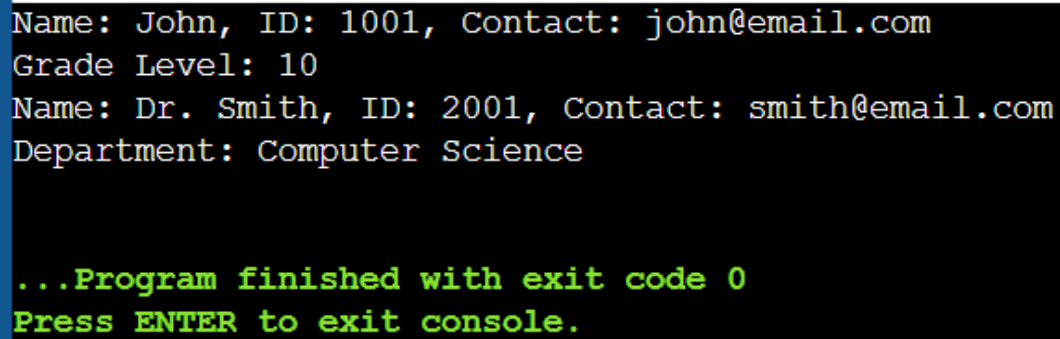
```

    Teacher t("Dr. Smith", 2001, "smith@email.com", "Computer Science");
    t.displayTeacher();

    return 0;
}

```

OUTPUT



```

Name: John, ID: 1001, Contact: john@email.com
Grade Level: 10
Name: Dr. Smith, ID: 2001, Contact: smith@email.com
Department: Computer Science

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 9

Implement multilevel inheritance with Vehicle -> Truck -> RefrigeratedTruck.

Code

```

#include <iostream>
#include <string>
using namespace std;

class Vehicle {
protected:
    string make;
    string model;
    int year;
public:
    Vehicle(string mk, string md, int y)
        : make(mk), model(md), year(y) {}
    void displayVehicle() {
        cout << "Make: " << make
            << ", Model: " << model
            << ", Year: " << year << endl;
    }
};

class Truck : public Vehicle {
protected:
    double loadCapacity;
public:

```

```

    Truck(string mk, string md, int y, double lc)
        : Vehicle(mk, md, y), loadCapacity(lc) {}
    void displayTruck() {
        displayVehicle();
        cout << "Load Capacity: " << loadCapacity << " tons" << endl;
    }
};

class RefrigeratedTruck : public Truck {
private:
    double temperatureControl;
public:
    RefrigeratedTruck(string mk, string md, int y, double lc, double tc)
        : Truck(mk, md, y, lc), temperatureControl(tc) {}
    void displayRefrigeratedTruck() {
        displayTruck();
        cout << "Temperature Control: " << temperatureControl << " C" << endl;
    }
};

int main() {
    RefrigeratedTruck rt("Volvo", "FH16", 2024, 40.0, -18.0);
    rt.displayRefrigeratedTruck();
    return 0;
}

```

OUTPUT

```

Make: Volvo, Model: FH16, Year: 2024
Load Capacity: 40 tons
Temperature Control: -18 C

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 10

Implement hybrid inheritance with Person -> Staff, Student and TeachingAssistant inheriting from both Staff and Student.

Code

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
    string address;
public:
    Person(string n, string a) : name(n), address(a) {}
    void displayPerson() {
        cout << "Name: " << name
              << ", Address: " << address << endl;
    }
};

class Staff : virtual public Person {
protected:
    int employeeId;
    string department;
public:
    Staff(string n, string a, int id, string d)
        : Person(n, a), employeeId(id), department(d) {}
    void displayStaff() {
        displayPerson();
        cout << "Employee ID: " << employeeId
              << ", Department: " << department << endl;
    }
};

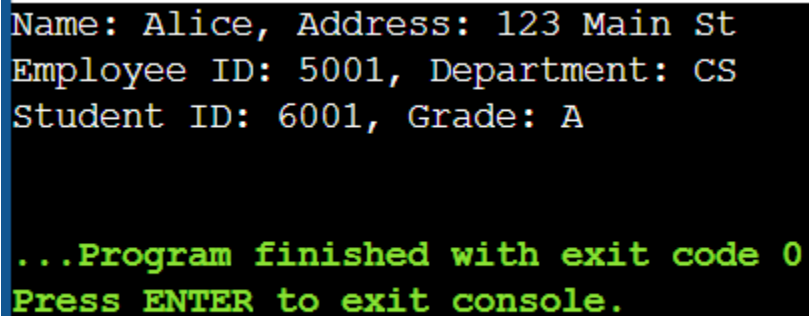
class Student : virtual public Person {
protected:
    int studentId;
    string grade;
public:
    Student(string n, string a, int id, string g)
        : Person(n, a), studentId(id), grade(g) {}
    void displayStudent() {
        displayPerson();
        cout << "Student ID: " << studentId
              << ", Grade: " << grade << endl;
    }
};

class TeachingAssistant : public Staff, public Student {
public:
    TeachingAssistant(string n, string a, int eid, string d, int sid, string g)
```

```
        : Person(n, a), Staff(n, a, eid, d), Student(n, a, sid, g) {}
void displayTA() {
    displayPerson();
    cout << "Employee ID: " << employeeId
          << ", Department: " << department << endl;
    cout << "Student ID: " << studentId
          << ", Grade: " << grade << endl;
}
};

int main() {
    TeachingAssistant ta("Alice", "123 Main St", 5001, "CS", 6001, "A");
    ta.displayTA();
    return 0;
}
```

OUTPUT

A screenshot of a terminal window with a black background. The output text is displayed in a monospaced font. The first three lines are in white: "Name: Alice, Address: 123 Main St", "Employee ID: 5001, Department: CS", and "Student ID: 6001, Grade: A". The last two lines are in green: "...Program finished with exit code 0" and "Press ENTER to exit console.".

Name: Alice, Address: 123 Main St
Employee ID: 5001, Department: CS
Student ID: 6001, Grade: A

...Program finished with exit code 0
Press ENTER to exit console.